

Algorithms and data structures II.

2021/2022/1

Contents

Contents	2
Data compression.....	3
Naive compression	3
Huffman codes	6
Lempel – Ziv – Welch (LZW) compression	9
Bibliography	13

Data compression

In informatics, coding theory deals with different representations of data and data transmission. One type of coding is source coding, which encodes the data to represent information more efficiently. It examines the length of the given form of data, that is, how much data can be stored with a given amount of information.

Source coding aims to represent information in a compact (shorter) form, so data compression is often called coding. Encoding and decoding are used to mean compression and decompression, respectively.

We usually compress information to save time and/or space. When transmitting information over a network, the fewer bits transmitted, the quicker the transmission. Therefore, the sender often compresses the data before sending it, sends the compressed data, and then decompresses the data that it receives. This can save a lot of time in many cases.

When the amount of storage available could limit how much information you can store, you can store more information if it's compressed. This is the reason why we want to save space.

Compression methods can be **lossless** or **lossy**. With **lossless compression**, when the compressed information is decompressed, it is identical to the original information. With lossy compression, the decompressed information differs from the original, but ideally in an insignificant manner. MP3 and JPEG compression are lossy, but the compression method used by the zip program is lossless. When compressing text, we want lossless compression. Even a difference of one bit can be meaningful. The following data compression methods are all lossless.

Since computers are binary-based, the **universal unit of information** is $r = 2$, and we will have binary codewords with the alphabet $T = \{0, 1\}$. The finite words with the alphabet T are codewords, and the **code** will be any nonempty subset of the codewords. The number of members in T is the size and is denoted $|T|$.

We can build a binary tree (coding tree) to represent the code expressively. This tree tells us how to form the codes and is convenient when decompressing. The tree nodes are words, and next to each edge in the tree appear the possible characters of the codewords as labels.

Coding by characters means that the data with the original **alphabet** Σ is encoded with the following bijective mapping: $\Sigma \rightarrow C \subset T^*$.

Naive compression

We code the input string character by character with **fixed-length code**. The code is even when the codewords have the same length. This method compresses the data using even code.

Let $\Sigma = \langle \sigma_1, \sigma_2, \dots, \sigma_d \rangle$ be the alphabet.

The coded data with the alphabet Σ will be as small as possible if the common length of the codewords is as small as possible. Since $|T| = r$ and $|\Sigma| = d$, we can code each character with a length $\lceil \log_r d \rceil$.

We can code each character with $\lceil \log_2 d \rceil$ bits, because with $\lceil \log_2 d \rceil$ bit $2^{\lceil \log_2 d \rceil}$ different binary code can be represented, and $2^{\lceil \log_2 d \rceil} \geq d > 2^{\lceil \log_2 d \rceil - 1}$, so with $\lceil \log_2 d \rceil$ bit we can represent d different code, but with $\lceil \log_2 d \rceil - 1$ bit we can not do that.

(Let $\log_2 d = \log d$; the default base will be 2, since $r = 2$ because of the alphabet $T = \{ '0', '1' \}$.)

In: $\Sigma \langle \rangle$ is the input string, $n = |In|$. It can be encoded with $n * \lceil \log d \rceil$ bits. (n is the length of the string, and d is the size of the alphabet.)

Example.

Compress the text: ABRAKADABRA

$d = 5$ and $n = 11$

→ the length of the compressed code: $11 * \lceil \log 5 \rceil = 11 * 3 = 33$ bit

(The codes with the size of 3 bits can represent the 5 characters fully optionally.)

The compressed file must include a decoding table with the processed information.

The decoding table of ABRAKADABRA:

character	code
A	000
B	001
D	010
K	011
R	100

Encoded form of ABRAKADABRA: 000001100000011000010000001100000.

In practice, it is a prevalent coding method when it is not important how effective the compression is. For example, the ASCII code is based on this coding algorithm with its 8-bit long codewords. Longer texts tend to yield better results because of the large size of the decoding table.

Huffman codes

Huffman codes compress data very effectively: savings of 20% to 90% are typical, depending on the data's characteristics. Huffman's greedy algorithm uses a table giving how often each character occurs (i.e., its frequency) to build up an optimal way of representing each character as a binary string.

If we use a **fixed-length code**, we need 3 bits to represent 6 characters: a = 000, b = 001, . . . , f = 101. Can we do better? A **variable-length code** can do considerably better than a fixed-length code by giving frequent characters short and infrequent long codewords.

If we measure the efficiency of compression methods according to the length of the compressed information, then Huffman codes are the best of the prefix-free codes. Once we know the character frequencies, Huffman's method builds a binary code tree. This tree tells us how to form the codes and is also convenient when decompressing.

The steps of the algorithm for binary coding are the following:

1. Read through the uncompressed text to determine the frequencies of the characters.
2. Let Q be an empty priority queue. Construct a new node z containing the character itself and its frequency as key for every character. Each node forms a one-element tree.
3. a) We find the two nodes with the lowest frequencies and create a new root with these nodes as its children.
b) Label the parent-children edges by 0 and 1 according to the frequencies.
c) Construct a new node for the parent node whose frequency is the sum of its children's frequencies.
4. Repeat step 3 if there is more than one node in the queue.
5. To determine the code for a character, follow the path in the tree from the root down to the character's leaf and concatenate the bits along the path.
6. Reread the input and code it character by character.

Steps of the decompression

1. Travel down the binary tree according to the bits of the compressed information, starting at the root.
2. Strip off each bit, going left if it's a 0 and right if it's a 1.
3. Upon arriving at a leaf, stop, emit a character, and resume searching at the root.

4. Decompression continues this way until all the bits of the compressed information have been processed.
- One disadvantage of the Huffman coding is that it requires the frequencies of all the characters to be known in advance, and therefore, compression requires two passes over the uncompressed text: one to determine character frequencies and one to map each character to its code.
 - The tree that the method builds is a **strictly binary tree**.
 - The compressed file represents the binary tree with the compressed information. If we already have the binary tree built before decompressing, it takes constant time to process each bit.

In the encoding we use, not only do the more frequent characters get the shorter bit sequences but there's something else interesting about the encodings: no code is a prefix of any other code.

The Huffman is a **prefix-free** code. If a code has the prefix-free property, it means that no word in the code is a prefix of any other word in the code: in other words, no longer code words begin with the identical digits making up any of the shorter code words. Because no code is a prefix of any other code, we can unambiguously match the compressed bits with their original characters as we decompress bits in order. If we draw a code tree for a prefix code, all code words will be located at leaf nodes in that tree.

In general, Huffman codes are not unambiguous. If there are more equal frequencies, we can get different Huffman codes. On the other hand, the labels themselves don't matter much. The 0 and 1 labels can be used inversely.

Since the coding is different for each given data, it must be known on the decoding side. One possibility is to include a binary tree representation with the compressed information. Another possibility is to include a **decoding table** with the processed data. Each entry of the table would include the character, the number of bits in its code, and the code itself. From this table, it's possible to build the binary tree in time linear in the total number of bits in all codes.

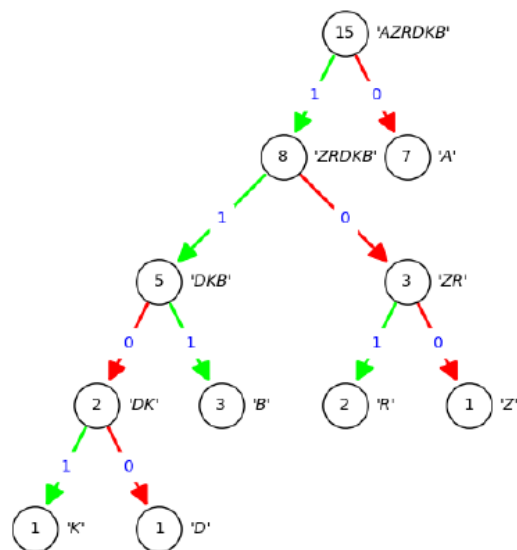
Example.

Compress the text: AZABBRAKADABRAA

character	frequency	code
A	7	0
B	3	111
D	1	1100
K	1	1101
R	2	101
Z	1	100

We construct the nodes in the minimum priority queue, containing the characters and their frequencies as labels, so we can build the binary tree.

<1 1 1 2 3 7>
D K Z R B A
<1 2 2 3 7>
Z DK R B A
<2 3 3 7>
DK B ZR A
<3 5 7>
ZR DKB A
<7 8>
A ZRDKB
<15>
AZRDKB



Encoded form of AZABBRAKADABRAA: 01000111111010110101100011110100.

Given the frequencies of the six characters, to encode the n-character (n=15) strand, we need only $7*1 + 3*3 + 1*4 + 1*4 + 2*3 + 1*3 = 33$ bits.

Lempel – Ziv – Welch (LZW) compression

We can easily find data whose representation can be more compact when we find codewords to character sequences and not only for characters. The **dictionary-based compression algorithms** take advantage of recurring information. The idea of the next compressing scheme we'll examine relies on reoccurring patterns to save data space. It is known as LZW and is the basis for many compression programs used in practice.

LZW makes a single pass over its input for compression and decompression. In both, it builds a dictionary of character sequences that it has seen and uses indexes to represent character sequences. We can think of the dictionary as a set of character strings; denote it S .

We can assume the following features of the dictionary:

- Both the compressor and decompressor seed the dictionary with a one-character sequence for each character in the character set.
- If a character string is already in the dictionary, then all its prefixes are also in the dictionary.
- The stored words have codes with the same length. The code for the word $x \in S$ is $c(x)$.

The dictionary does not have to be stored with the compressed information. LZW decompression rebuilds the dictionary directly from the compressed information. In practice, the optimal length of the codes is 12-15 bits.

The compression manages two variables: s is a string, and c is a character. Initially, s is just the first character of the input.

The general step of the algorithm:

- (1) Read the next character of the input into the variable c .
- (2) If the reading attempt was unsuccessful (it means that we reached the end of the input), then output the code of s from the dictionary.
- (3) If sc is in the dictionary, set s to sc . The compressor is building a longer string that will eventually be inserted into the dictionary. Go back to step 1.
- (4) Otherwise, when s is in the dictionary but sc is not, the compressor outputs the index of s in the dictionary, inserts sc into the next available dictionary entry, and sets s to just the input character c . Go back to step 1.

Example.

Let the character string be: ABABABAACAACCBBAAAAAAAAAA

The dictionary at the start of the algorithm:

character	code
A	1
B	2
C	3

During the compression, we make a single pass over the input and look for the longest word that we already know (it is in the dictionary and has a code) in every step. When we found it, then

- put the code of the word we just found into the output,
- and Insert sc into the next available dictionary entry, where s is the longest known string and c is the next character

At the beginning, the longest word seen is A, whose code is 1. It is the first character of the input. Then, the algorithm reads the next character from the input and, since AB is not in the dictionary, inserts it into the next available entry, which is the one with code 4. We repeat the general step until we reach the end of the input.

code	current word	next character	new code
1	A	B	4
2	B	A	5
4	AB	A	6
6	ABA	A	7
1	A	C	8
3	C	A	9
1	A	A	10
8	AC	C	11
3	C	B	12
2	B	B	13
5	BA	A	14
10	AA	A	15
15	AAA	A	16
15	AAA	-	-

The encoded message: 1 2 4 6 1 3 1 8 3 2 5 10 15 15

Like the compressor, the decompressor seeds the dictionary with the one-character sequences for each character in the character set. The dictionary can be seen below. We can fill the first and last columns of the table, and then we continue filling the table row by row.

Since the first character of the word in the second row is the same as the next character in the first row, it is known. So now we can see the new word coded in the first step at the compressing (AB).

code	current word	next character	new code
1	A	B	4
2	B	A	5
4	AB	A	6
6			7
...	...	...	...

Most of the time, the following dictionary index in the input is for an entry already in the dictionary. So, the LZW decompressor finds the string at that index in the dictionary and outputs it. Let's see the case when s is in the dictionary, but the string sc is not. The compressor outputs the index of s in the dictionary, inserts sc into the dictionary, and starts building a new string to store, starting with c . The decompressor has to match this behaviour.

For each index LZW takes from its input, it outputs the string s at that index in the dictionary. But it also knows that when the compressor outputs the index for s , the compressor does not have the string sc in the dictionary, where c is the character immediately following s . The decompressor knows that the compressor inserted the string sc into the dictionary.

It cannot insert sc yet because it hasn't seen the character c . That's coming as the first character of the next string that the decompressor will output. But the decompressor doesn't have that next string just yet. Therefore, the decompressor must keep track of two consecutive strings of its output. If the decompressor outputs strings X and Y in that order, it concatenates Y 's first character onto X and then inserts the resulting string into the dictionary.

We can now easily finish the decompression. The dictionary will be the same as it was at the compression, and we can read the decoded string from the second row.

The method compresses long texts, but not that well. Part of the problem stems from the extensive dictionary, searching takes almost as long as reading through the original text. This is why we have to limit the set of codewords. For example

- we can limit the maximum number of the codewords;
- we can limit the max length of the codewords;
- or we build the dictionary on a given input prefix and then use it to compress the rest of the text.

Bibliography

THOMAS H. CORMEN, CHARLES E. LEISERSON, RONALD L. RIVEST, CLIFFORD STEIN:
Introduction to algorithms, 2003

THOMAS H. CORMEN: Algorithms Unlocked, 2013

IVANYOS GÁBOR, RÓNYAI LAJOS, SZABÓ RÉKA: Algoritmusok

NAGY ÁDÁM: Algoritmusok és adatszerkezetek II. gyakorlati segédlet - Tömörítés